

Experiment-3

Name: -P.Divyashree

Regno: -192472291

Course Code: -MLA0305

Slot-B

Policy and Value Function Representation Using Bellman Equations

Aim: -To implement and represent Policy Functions and Value Functions using the Bellman Expectation Equation and Bellman Optimality Equation in a Reinforcement Learning environment.

Procedure

1. Define the states and possible actions of the environment.
2. Initialize a policy for selecting actions in different states.
3. Assign rewards and transition probabilities to the environment.
4. Initialize the value function $V(s)$ for all states.
5. Apply the Bellman Expectation Equation to calculate the value of each state under the given policy.
6. Update the state values iteratively until the value function converges.
7. Calculate action values for the available actions.
8. Apply the Bellman Optimality Equation to determine the maximum expected value for each state.
9. Select the action having the highest expected return.
10. Represent the resulting policy and value function.
11. Compare the values obtained before and after convergence.
12. Visualize the state values and policy to understand the relationship between policy and value functions.

Source Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
ROWS = 5
COLS = 5
GAMMA = 0.90
THETA = 0.001
MAX_ITERATIONS = 1000
states = [(r, c) for r in range(ROWS) for c in range(COLS)]
GOAL = (4, 4)
OBSTACLES = [(1, 1), (2, 3), (3, 1)]
ACTIONS = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
```

```
}
```

```
ACTION_SYMBOLS = {
```

```
    "UP": "↑",
```

```
    "DOWN": "↓",
```

```
    "LEFT": "←",
```

```
    "RIGHT": "→"
```

```
}
```

```
def get_next_state(state, action):
```

```
    """
```

```
    Returns the next state and reward.
```

```
    Reward structure:
```

```
        +10 -> reaching goal
```

```
        -5  -> hitting obstacle
```

```
        -1  -> normal movement
```

```
        -2  -> hitting boundary
```

```
    """
```

```
    # Goal is terminal
```

```
    if state == GOAL:
```

```
        return state, 0
```

```
    dr, dc = ACTIONS[action]
```

```
    new_r = state[0] + dr
```

```
    new_c = state[1] + dc
```

```
    # Boundary condition
```

```
    if new_r < 0 or new_r >= ROWS or new_c < 0 or new_c >= COLS:
```

```
        return state, -2
```

```
    next_state = (new_r, new_c)
```

```
    # Obstacle
```

```
    if next_state in OBSTACLES:
```

```
        return state, -5
```

```
    # Goal
```

```
    if next_state == GOAL:
```

```
        return next_state, 10
```

```
    # Normal movement
```

```
    return next_state, -1
```

```

V = {}

for state in states:
    V[state] = 0.0

# Obstacles are not considered normal states
for obstacle in OBSTACLES:
    V[obstacle] = np.nan

# Goal value
V[GOAL] = 10.0

# -----
# 4. VALUE ITERATION USING BELLMAN OPTIMALITY EQUATION
# -----

iteration_values = []
iteration_deltas = []

print("\n" + "=" * 70)
print("    VALUE ITERATION USING BELLMAN EQUATION")
print("=" * 70)

for iteration in range(1, MAX_ITERATIONS + 1):

    new_V = V.copy()

    delta = 0

    for state in states:

        # Skip obstacle
        if state in OBSTACLES:
            continue

        # Goal state
        if state == GOAL:
            new_V[state] = 10.0
            continue

    action_values = []

    # Calculate Q(s,a) for every action
    for action in ACTIONS:

```

```

    next_state, reward = get_next_state(state, action)

    # Bellman equation
    value = reward + GAMMA * V[next_state]

    action_values.append(value)

    # Bellman optimality equation
    new_V[state] = max(action_values)

    delta = max(delta, abs(new_V[state] - V[state]))

V = new_V

# Store values for visualization
iteration_values.append(
    np.mean([
        value for state, value in V.items()
        if state not in OBSTACLES
    ])
)

iteration_deltas.append(delta)

# Display progress
if iteration <= 10 or iteration % 10 == 0:
    print(
        f"Iteration {iteration:3d} | "
        f"Delta = {delta:.6f}"
    )

# Convergence
if delta < THETA:
    print("\nValue Iteration Converged!")
    print(f"Converged after {iteration} iterations.")
    break

# -----
# 5. EXTRACT OPTIMAL POLICY
# -----

policy = {}

```

```

for state in states:

    if state in OBSTACLES:
        policy[state] = "OBSTACLE"
        continue

    if state == GOAL:
        policy[state] = "GOAL"
        continue

    best_action = None
    best_value = -np.inf

    for action in ACTIONS:

        next_state, reward = get_next_state(state, action)

        action_value = reward + GAMMA * V[next_state]

        if action_value > best_value:
            best_value = action_value
            best_action = action

    policy[state] = best_action

```

```

# -----
# 6. CREATE VALUE FUNCTION MATRIX
# -----

```

```

value_matrix = np.zeros((ROWS, COLS))

```

```

for r in range(ROWS):
    for c in range(COLS):

        state = (r, c)

        if state in OBSTACLES:
            value_matrix[r, c] = np.nan

        else:
            value_matrix[r, c] = V[state]

```

```

# -----

```

7. CREATE POLICY MATRIX

```
policy_matrix = []
```

```
for r in range(ROWS):
```

```
    row = []
```

```
    for c in range(COLS):
```

```
        state = (r, c)
```

```
        if state in OBSTACLES:
```

```
            row.append("■ ")
```

```
        elif state == GOAL:
```

```
            row.append("G")
```

```
        else:
```

```
            row.append(
                ACTION_SYMBOLS[policy[state]]
            )
```

```
    policy_matrix.append(row)
```

8. SUMMARY RESULTS

```
normal_states = [
```

```
    state for state in states
```

```
    if state not in OBSTACLES
```

```
]
```

```
values = [
```

```
    V[state] for state in normal_states
```

```
    if state != GOAL
```

```
]
```

```
summary = pd.DataFrame({
```

```
    "Metric": [
```

```
        "Grid Size",
```

```
        "Total States",
```

```

        "Obstacle States",
        "Goal State",
        "Discount Factor (Gamma)",
        "Convergence Threshold",
        "Iterations",
        "Maximum State Value",
        "Minimum State Value",
        "Average State Value",
        "Goal Value"
    ],

    "Result": [
        f"{ROWS} x {COLS}",
        len(states),
        len(OBSTACLES),
        str(GOAL),
        GAMMA,
        THETA,
        iteration,
        round(max(values), 3),
        round(min(values), 3),
        round(np.mean(values), 3),
        V[GOAL]
    ]
})

print("\n" + "=" * 70)
print("          FINAL VALUE FUNCTION")
print("=" * 70)

for r in range(ROWS):

    row_values = []

    for c in range(COLS):

        state = (r, c)

        if state in OBSTACLES:
            row_values.append(" BLOCK ")

        else:
            row_values.append(
                f"{V[state]:7.2f}"
            )

```

```

    print(" | ".join(row_values))

print("\n" + "=" * 70)
print("          OPTIMAL POLICY")
print("=" * 70)

for row in policy_matrix:
    print(" | ".join(f"{x:^7}" for x in row))
print("\n" + "=" * 70)
print("          SUMMARY RESULTS")
print("=" * 70)

print(summary.to_string(index=False))
plt.figure(figsize=(10, 8))

sns.heatmap(
    value_matrix,
    annot=True,
    fmt=".2f",
    cmap="YlGnBu",
    linewidths=2,
    linecolor="white",
    cbar_kws={
        "label": "State Value V(s)"
    }
)

plt.title(
    "Value Function Heatmap\n"
    "Bellman Optimality Equation",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Column / State Position")
plt.ylabel("Row / State Position")

plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 8))

plt.imshow(
    np.nan_to_num(
        value_matrix,

```



```

        nan=np.nanmin(value_matrix)
    ),
    cmap="YlGnBu"
)

for r in range(ROWS):

    for c in range(COLS):

        state = (r, c)

        if state in OBSTACLES:

            plt.text(
                c,
                r,
                "BLOCK",
                ha="center",
                va="center",
                fontsize=11,
                fontweight="bold"
            )

        elif state == GOAL:

            plt.text(
                c,
                r,
                "GOAL",
                ha="center",
                va="center",
                fontsize=12,
                fontweight="bold"
            )

        else:

            plt.text(
                c,
                r,
                ACTION_SYMBOLS[policy[state]],
                ha="center",
                va="center",
                fontsize=25,
                fontweight="bold"
            )

```

```

    )

    plt.text(
        c,
        r + 0.32,
        f"{V[state]:.1f}",
        ha="center",
        va="center",
        fontsize=8
    )

plt.title(
    "Optimal Policy with State Values",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Column")
plt.ylabel("Row")

plt.xticks(range(COLS))
plt.yticks(range(ROWS))

plt.colorbar(
    label="State Value V(s)"
)

plt.grid(
    linewidth=1,
    alpha=0.3
)

plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 6))

plt.plot(
    range(1, iteration + 1),
    iteration_deltas,
    marker="o",
    linewidth=2
)

plt.axhline(

```

```

    THETA,
    linestyle="--",
    linewidth=2,
    label="Convergence Threshold"
)

plt.title(
    "Bellman Value Iteration Convergence",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Iteration")
plt.ylabel("Maximum Change (Delta)")

plt.legend()
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
state_labels = [
    f"S{r}{c}"
    for r, c in normal_states
]

state_values = [
    V[state]
    for state in normal_states
]

plt.figure(figsize=(14, 6))

bars = plt.bar(
    state_labels,
    state_values
)

plt.axhline(
    0,
    linewidth=1
)

plt.title(
    "State Value Comparison",
    fontsize=16,

```

```

        fontweight="bold"
    )
    plt.xlabel("State")
    plt.ylabel("Value V(s)")
    plt.xticks(rotation=45)
    for bar, value in zip(bars, state_values):

        plt.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height(),
            f"{value:.2f}",
            ha="center",
            va="bottom",
            fontsize=8
        )
    plt.grid(
        axis="y",
        alpha=0.3
    )
    plt.tight_layout()
    plt.show()
    plt.figure(figsize=(10, 6))
    plt.plot(
        range(1, iteration + 1),
        iteration_values,
        marker="o",
        linewidth=2
    )
    plt.title(
        "Average State Value During Value Iteration",
        fontsize=16,
        fontweight="bold"
    )

    plt.xlabel("Iteration")
    plt.ylabel("Average V(s)")

    plt.grid(alpha=0.3)

    plt.tight_layout()
    plt.show()

```

Sample Dataset (10 Samples): -

State	Action	Next_State	Reward
(0, 0)	RIGHT	(0, 1)	-1
(0, 1)	RIGHT	(0, 2)	-1
(0, 2)	DOWN	(1, 2)	-1
(1, 2)	DOWN	(2, 2)	-1
(2, 2)	RIGHT	(2, 2)	-5
(2, 2)	DOWN	(3, 2)	-1
(3, 2)	DOWN	(4, 2)	-1
(4, 2)	RIGHT	(4, 3)	-1
(4, 3)	RIGHT	(4, 4)	10
(4, 4)	UP	(4, 4)	0

FIG: -1 SAMPLE DATA SET

Results

=====								
FINAL VALUE FUNCTION								
=====								
3.87		5.41		7.12		9.03		11.14
5.41		BLOCK		9.03		11.14		13.49
7.12		9.03		11.14		BLOCK		16.10
9.03		BLOCK		13.49		16.10		19.00
11.14		13.49		16.10		19.00		10.00

FIG: -2 FINAL VALUE FUNCTION

```

=====
                        FINAL RESULT
=====

Algorithm                : Value Iteration

Bellman Equation          :  $V(s) = \max [R + \text{Gamma} * V(s')]$ 

Discount Factor           : 0.9

Convergence Threshold     : 0.001

Iterations Completed      : 9

Sample Dataset Size       : 10

Goal State                : (4, 4)

Goal Value                : 10.00

Maximum State Value       : 19.00

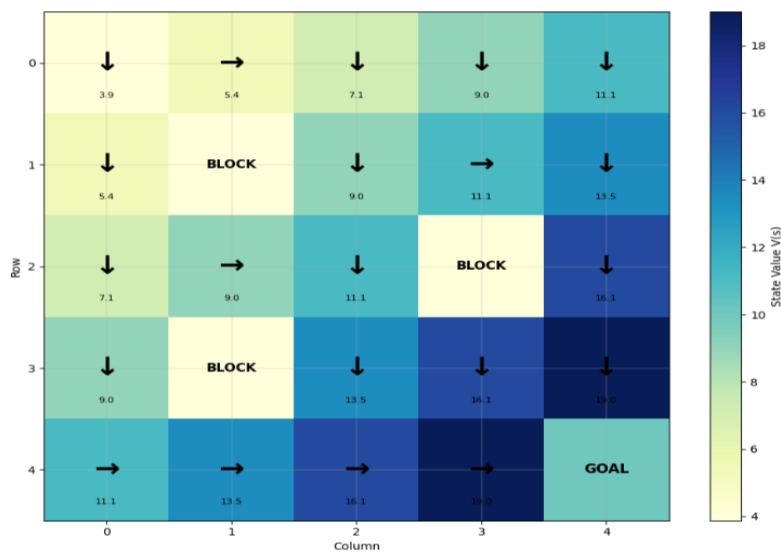
Minimum State Value       : 3.87

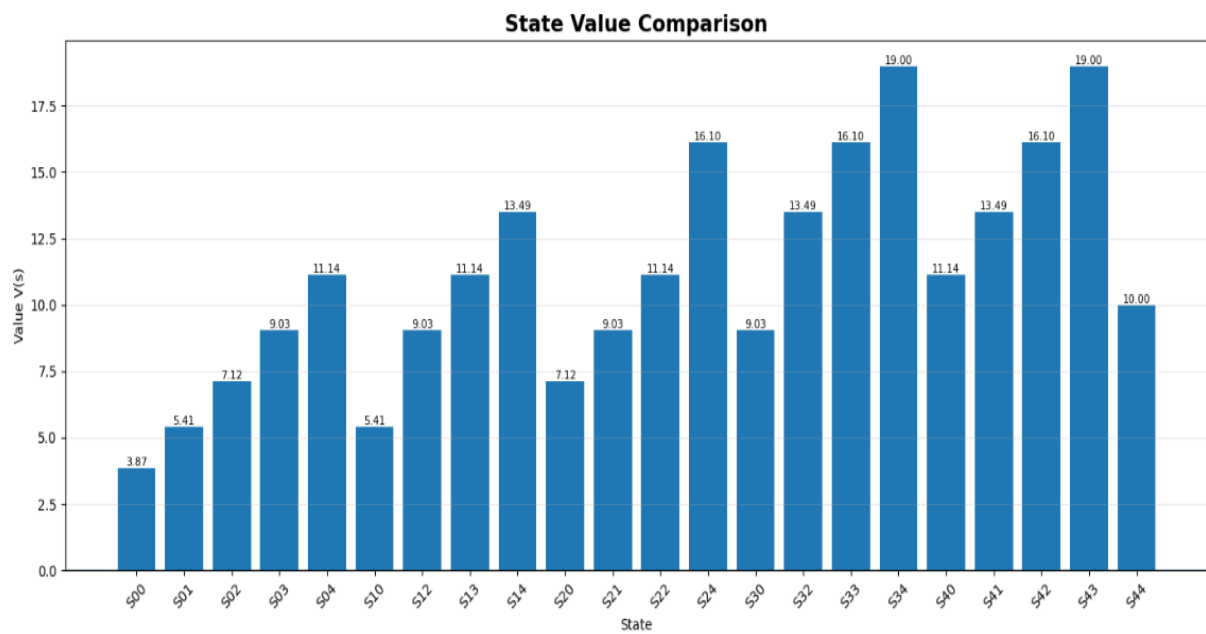
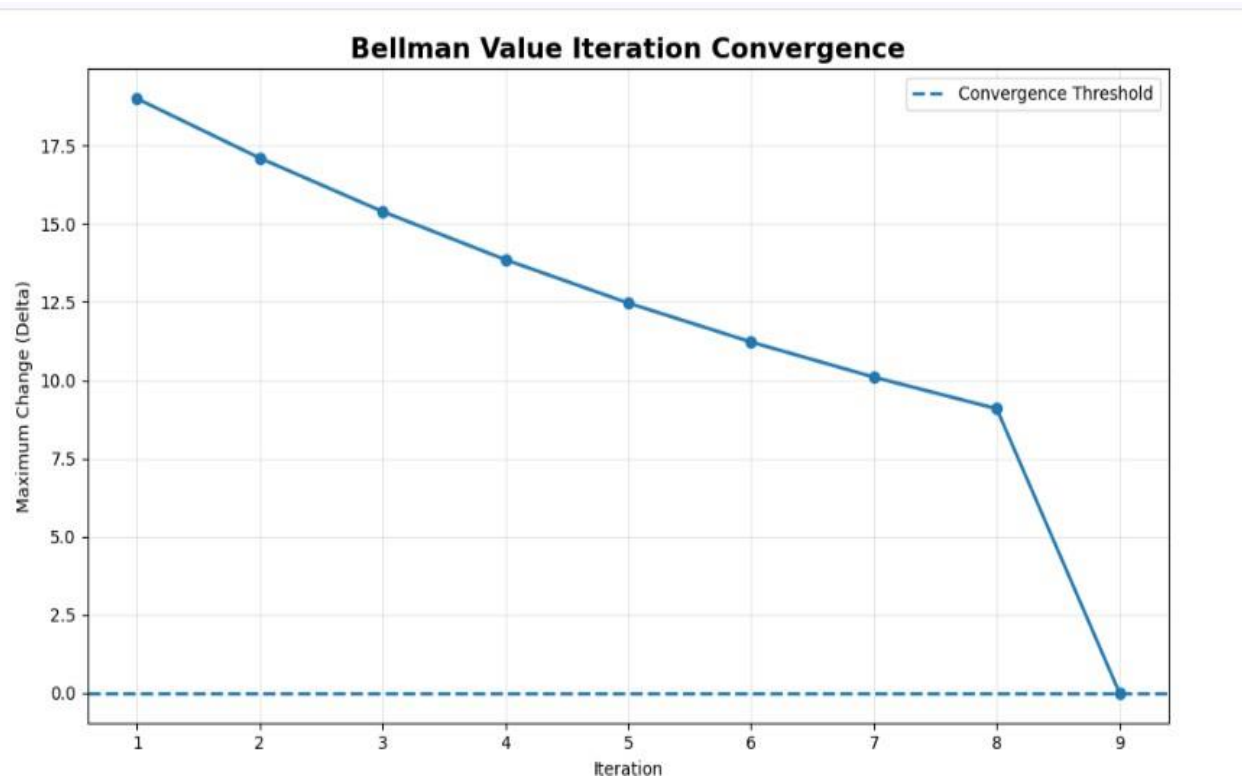
Average State Value       : 11.26

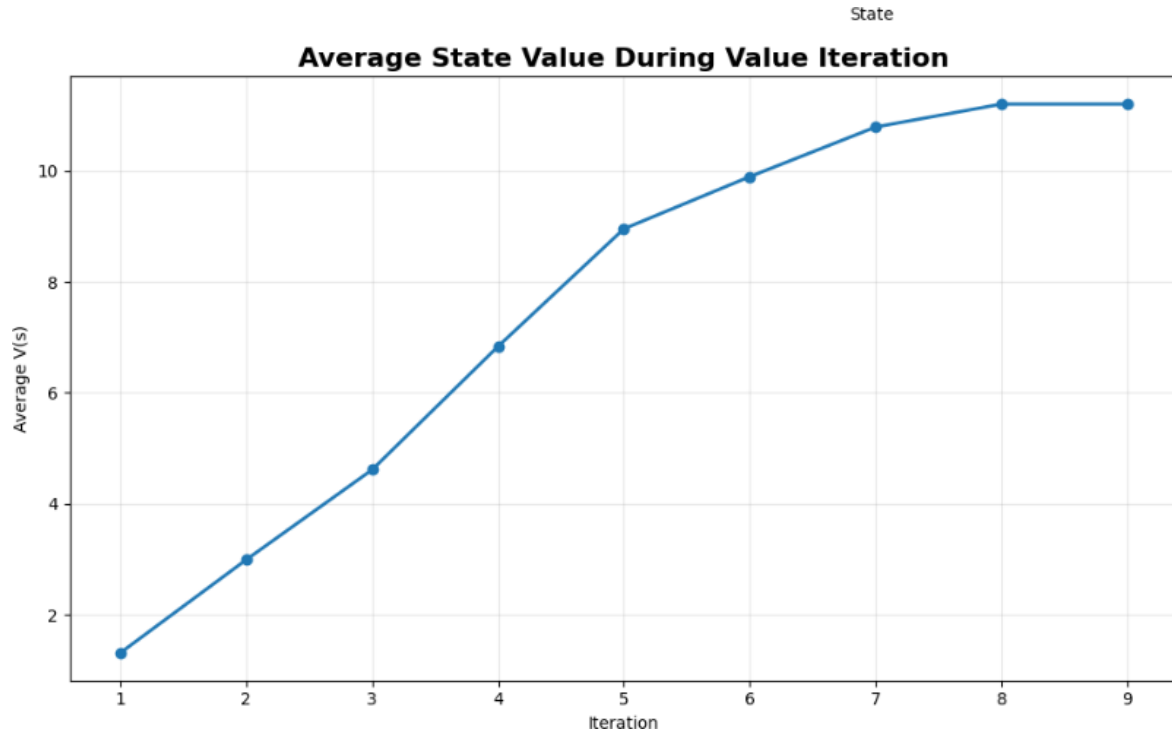
```

FIG: -3 RESULTS

VISUALIZATIONS: -







Results: - Thus the implementation of Policy Functions and Value Functions using the Bellman Expectation Equation and Bellman Optimality Equation in a Reinforcement Learning environment has been executed Successfully.